

Unit 5 – Part 1

Introduction to Multithread Programming

What is a Thread?

- A **thread** is a lightweight sub-process, the smallest unit of processing.
- It is a separate path of execution in a program.
- Multiple threads can run **concurrently** in a single process.

What is Multithreading?

- **Multithreading** is a programming concept where multiple threads run **simultaneously** to perform tasks in parallel.
- Improves performance, especially in tasks like:
 - Handling multiple user requests
 - Background tasks
 - Games and simulations
 - Real-time systems

Benefits of Multithreading

- Efficient CPU utilization
 - Simultaneous task execution
 - Improved application responsiveness
 - Resource sharing between threads (memory, files, etc.)
-

Thread Class

Creating a Thread by Extending Thread Class

```
class MyThread extends Thread
{
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args)
    {
        MyThread t1 = new MyThread();
        t1.start(); // starts a new thread
    }
}
```

Important Methods of Thread Class

Method	Description
start()	Starts the execution of the thread
run()	Contains the code to be executed
sleep(ms)	Pauses the thread for specified time
join()	Waits for the thread to finish
isAlive()	Checks if thread is alive or not

Runnable Interface

Creating a Thread by Implementing Runnable Interface

```
class MyRunnable implements
    Runnable { public void run() {
        System.out.println("Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args)
    { MyRunnable obj = new
      MyRunnable();
      Thread t1 = new Thread(obj); // pass Runnable to Thread constructor
      t1.start();
    }
}
```

✓ Why Use Runnable?

- Useful when you want to extend another class (Java supports only single inheritance).
- More flexible and better resource sharing.

Key Differences: Thread vs. Runnable

Aspect	Thread Class	Runnable Interface
Inheritance	Extends Thread class	Implements Runnable interface
Flexibility	Less flexible (can't extend another class)	More flexible
Object Sharing	Difficult	Easier
Preferred When	No need to extend other class	Need to extend other classes

Thread Priority

What is Thread Priority?

- Each thread in Java has a **priority** that helps the **thread scheduler** decide the order of execution.
- Priorities range from **1 (MIN_PRIORITY)** to **10 (MAX_PRIORITY)**.
- Default priority** is **5 (NORM_PRIORITY)**.

Priority Constants:

Constant	Value
Thread.MIN_PRIORITY	1
Thread.NORM_PRIORITY	5
Thread.MAX_PRIORITY	10

Setting Priority:

```
Thread t1 = new Thread();
t1.setPriority(Thread.MAX_PRIORITY); // sets to 10
```

Getting Priority:

```
int p = t1.getPriority();
System.out.println(p);
```

Note: Thread priorities are just hints to the scheduler. Actual behavior may vary by JVM and OS.

Synchronization

Synchronization is the mechanism that ensures **only one thread** can access a **critical section** of code or resource at a time. It prevents **race conditions**, ensuring **data consistency** in multi-threaded applications.

Why Synchronization?

When multiple threads access and modify **shared resources** (like variables, objects, files, etc.) **concurrently**, there is a risk of inconsistent or corrupt data. Synchronization helps in:

- Preventing **data inconsistency**
 - Avoiding **thread interference**
 - Managing **thread safety**
-

Types of Synchronization in Java

1. Synchronized Method

A whole method is locked so only one thread can execute it at a time.

```
synchronized void display() {  
    // critical section  
}
```

2. Synchronized Block

Only a specific part of code is locked on a given object or resource.

```
void show() {  
    synchronized(this)  
    {  
        // critical section  
    }  
}
```

How It Works:

Every object in Java has a built-in **monitor lock**. When a thread enters a synchronized method/block:

- It **acquires the lock** for that object.
- Other threads trying to access any synchronized code on that object are **blocked** until the lock is released.

Example: Without and With Synchronization

Without Synchronization:

```
class Counter {  
    int count = 0;  
    void increment() {  
        count++;  
    }  
}
```

With Synchronization:

```
class Counter  
{  
    int count = 0;  
    synchronized void  
        increment() { count++;  
    }  
}
```

Important Points:

- Synchronization affects **performance** due to overhead of acquiring locks.
- Use synchronization only for **critical sections**.
- Prefer using **synchronized blocks** for finer control.

Common Issues Without Synchronization

- **Race Condition:** Two threads read and write shared data, causing unpredictable results.
- **Thread Interference:** One thread interrupts the operation of another.

✓ What is the use of Synchronization?

- In **multithreading**, multiple threads may try to access shared resources at the same time.
- **Synchronization** is used to prevent **data inconsistency** and ensure **thread safety**.

Example Problem:

```
class Main {
    public static void main(String[] args) {
        Table table=new Table();
        A x=new A(table,10);
        B y=new B(table,5);

        x.start();
        y.start();
    }
}

class Table{
    synchronized void multiply(int n){
        for(int i=1;i<=10;i++)
            System.out.println(n+"*"+i+"="+n*i);
    }
}

class A extends Thread{
    Table table;
    int number;
    A(Table table,int number){
        this.table=table;
        this.number=number;
    }
    public void run(){
        table.multiply(this.number);
    }
}

class B extends Thread{
    Table table;
    int number;
    B(Table table,int number){
        this.table=table;
        this.number=number;
    }
    public void run(){
        table.multiply(this.number);
    }
}
```

**Output without
synchronized**

5*1=5
5*2=10
5*3=15
5*4=20
5*5=25
10*1=10
10*2=20
5*6=30
5*7=35
5*8=40
5*9=45
5*10=50
10*3=30
10*4=40
10*5=50

**Output with
synchronized**

10*1=10
10*2=20
10*3=30
10*4=40
10*5=50
10*6=60
10*7=70
10*8=80
10*9=90
10*10=100
5*1=5
5*2=10
5*3=15
5*4=20
5*5=25

Example:

```
public class Main{
    public static void main(String h[]){
        Counter counter=new Counter();

        A x=new A(counter);
        A x2=new A(counter);

        x.start();
        x2.start();
        try {
            x.join();
            x2.join();
        }
        catch (InterruptedException e) {
            System.out.println("Error: "+e);
        }
    }
}
```

```
System.out.println("Final count: " + counter.getcount()
);
}}
```

```
class Counter{
    private int count=0;

    void increment(){

        count++;

    }
    int getcount(){ return
        this.count;
    }
}
```

```
class A extends Thread{
    Counter counter;
    A(Counter counter){
        this.counter=counter;
    }
    public void run(){
        for(int i=0;i<100000;i++){
            counter.increment();
        }
    }
}
```

**Output without
synchronized**

Final count: 196765

**Output with
synchronized**

Final count: 200000

Key Points:

Concept	Description
synchronized keyword	Ensures only one thread accesses a block/method at a time
Race Condition	When threads interfere with each other while accessing shared data
Thread-safe code	Code that runs correctly when accessed by multiple threads concurrently

Thread Communication

(Inter Thread Communication)

What is Inter-Thread Communication?

Inter-thread communication is the process that allows multiple threads to **cooperate** and **coordinate** with each other by **sharing information** and **synchronizing execution**. It's primarily used to avoid **busy waiting** and to enable **thread cooperation** in a multi-threaded environment.

Why is it Needed?

When multiple threads are working together (e.g., producer-consumer problem), one thread may need to wait for another to complete a task. Instead of continuously checking (polling), inter-thread communication allows threads to **pause** and **resume** execution **efficiently**.

Important Methods (from java.lang.Object):

These methods can only be called from **synchronized context** (i.e., inside synchronized blocks or methods).

Method Description

- `wait()` Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()` on the same object.
- `notify()` Wakes up a single thread that is waiting on the object's monitor.
- `notifyAll()` Wakes up all threads that are waiting on the object's monitor.

Producer – Consumer problem:

```
class ProducerConsumer
{
    public static void main(String[] args)
    {
        Items obj = new Items();
        Consumer t1 = new Consumer(obj);
        Producer t2 = new Producer(obj);
        t1.start();
        t2.start();
    }
}

class Items
{
    int count = 1000;

    synchronized void consume(int cnt)
    {
        System.out.println("Items :"+this.count);
        System.out.println("Consumer : Going to consume "+cnt);
        if (this.count < cnt)
        {
            System.out.println("Low : waiting for produce..");
            try
            {
                { wait(); }
            }
            catch (InterruptedException e)
            { }
        }
        this.count = this.count - cnt;
        System.out.println("Consume completed");
        System.out.println("Current :"+this.count); }

    synchronized void produce(int cnt)
    {
        System.out.println("Producer : Going to produce "+cnt);
        this.count = this.count + cnt;
        System.out.println("Current :"+this.count);
        System.out.println("Produce Completed");
        notify();
    }
}
```

```
// creating a Thread1
class Consumer extends Thread
{
    Items pt;
    Consumer(Items pt)
    {
        this.pt = pt;
    }
    public void run()
    {
        pt.consume(1500);
    }
}

// creating a Thread2
class Producer extends Thread
{
    Items pt;
    Producer(Items pt)
    {
        this.pt = pt;
    }
    public void run()
    {
        pt.produce(800);
    }
}
```

Explanation of Above Code

This program creates two threads:

- A **Consumer** thread that tries to consume 1500 items.
- A **Producer** thread that produces 800 items.

These threads work on a shared resource Items, and synchronization ensures that the operations on this shared data (count) are thread-safe.

Class-by-Class Explanation

1. class ProducerConsumer

This is the **main class** with the main() method.

```
Items obj = new Items();
```

Consumer t1 = new Consumer(obj); Producer t2 =
new Producer(obj);

- A shared object obj of type Items is created.
 - Consumer and Producer threads are created and passed this shared object.
 - Both threads are started using start().
-

2. class Items

This class holds the shared **inventory** and the core logic.

int count = 1000; // initial item count

a. synchronized void consume(int cnt)

Handles **consumption of items**.

```
while (this.count < cnt) {  
    wait(); // Waits for more items to be produced  
}  
this.count = this.count - cnt;
```

- If requested items > available (count), the thread **waits**.
- Once enough items are available, it subtracts from count.

b. synchronized void produce(int cnt)

Handles **production of items**.

```
this.count = this.count + cnt; notify(); // Wakes up a  
waiting thread
```

- Adds cnt to the existing count.
- Calls notify() to wake up the waiting consumer.

A Both methods are locked and synchronized to prevent race conditions.

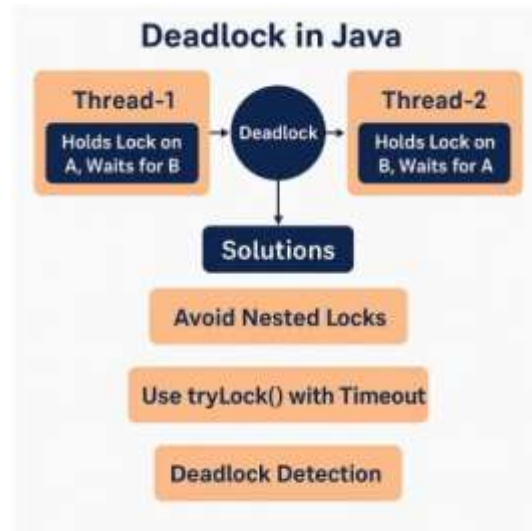
Thread Interaction Summary

1. **Initial count = 1000**
 2. **Consumer** tries to consume 1500:
 - Not enough items → goes into wait().
 3. **Producer** produces 800:
 - Updates count = 1000 + 800 = 1800
 - Calls notify() → wakes up Consumer.
 4. **Consumer** resumes, now has enough items:
 - count = 1800 - 1500 = 300
-

Deadlock

What is Deadlock?

A **deadlock** is a situation where two or more threads are **permanently blocked**, waiting for each other to release resources. It occurs when multiple threads hold **locks on different resources** and each thread waits to acquire a lock on the resource held by the other.



Example of Deadlock

```

class A {
    synchronized void methodA(B b) {
        System.out.println("Thread-1: Holding A, waiting for B...");
        synchronized (b) {
            System.out.println("Thread-1: Acquired B");
        }
    }
}

class B {
    synchronized void methodB(A a) {
        System.out.println("Thread-2: Holding B, waiting for A...");
        synchronized (a) {
            System.out.println("Thread-2: Acquired A");
        }
    }
}
    
```

- If **Thread-1** locks A and waits for B, and **Thread-2** locks B and waits for A, both threads wait **forever** — this is a deadlock.

Conditions for Deadlock

Deadlock typically happens when **all four** of the following conditions hold:

1. **Mutual Exclusion** – At least one resource is non-shareable.
2. **Hold and Wait** – A thread holds one resource and waits for another.
3. **No Preemption** – Resources can't be forcibly taken.
4. **Circular Wait** – A circular chain of threads exists, each waiting on the next.

Solutions to Deadlock

1. Avoid Nested Locks (Lock Ordering)

- Always acquire locks in a **fixed global order**.

```
synchronized (resource1) {  
    synchronized (resource2) {  
        // do work  
    }  
}
```

Ensure that all threads lock resource1 before resource2.

2. Minimize Synchronized Block Size

- Lock only the minimum amount of code necessary.

```
void safeMethod() {  
    // logic before sync  
    synchronized (this) {  
        // only critical code  
    }  
    // logic after sync  
}
```

Example:

```
class Common_resource{
    final String res1="Printer";
    final String res2="Scanner";
}

class Process1 extends Thread{
    Common_resource cr;

    Process1(Common_resource cr){
        this.cr=cr;
    }
    public void run(){
        synchronized(cr.res1){
            System.out.println(Thread.currentThread().getName()+" :Locked "+cr.res1);
            try{
                Thread.sleep(100);
            }
            catch(Exception e){

            }
        }
        synchronized(cr.res2){
            System.out.println(Thread.currentThread().getName()+" :Locked "+cr.res2);
        }
    }
}

class Process2 extends Thread{
    Common_resource cr;

    Process2(Common_resource cr){
        this.cr=cr;
    }
    public void run(){
        synchronized(cr.res2){
            System.out.println(Thread.currentThread().getName()+" :Locked "+cr.res2);
            try{
```

```
        Thread.sleep(100);
    }
    catch(Exception e){

    }
    synchronized(cr.res1){
        System.out.println(Thread.currentThread().getName()+" :Locked "+cr.res1);
    }
}

public class Locking_Resource{
    public static void main(String r[]){
        Common_resource cr=new Common_resource();

        Process1 p1=new Process1(cr);
        p1.setName("MS Word");
        Process2 p2=new Process2(cr);
        p2.setName("MS Excel");

        p1.start();
        p2.start();
    }
}
```

Explanation:

This program creates:

- A shared class Common_resource with two string resources: "Printer" and "Scanner".
- Two threads (Process1 and Process2) trying to lock these resources **in opposite order**.

This setup may lead to a **deadlock**, where both threads wait forever for each other to release a lock.

class Common_resource

```
final String res1 = "Printer"; final
String res2 = "Scanner";
```


- Defines two shared resources.
 - These are the objects that threads will try to **lock** using synchronized.
-

class Process1

```
synchronized(cr.res1){  
    // Locks res1  
    Thread.sleep(100); // Simulates delay  
    synchronized(cr.res2) {  
        // Tries to lock res2  
    }  
}
```

- Locks res1 first, then after a delay, tries to lock res2.
-

class Process2

```
synchronized(cr.res2){  
    // Locks res2  
    Thread.sleep(100); // Simulates delay  
    synchronized(cr.res1) {  
        // Tries to lock res1  
    }  
}
```

- Locks res2 first, then after a delay, tries to lock res1.
-

public class Locking_Resource

```
Common_resource cr = new Common_resource(); Process1 p1  
= new Process1(cr);  
Process2 p2 = new Process2(cr); p1.start();  
p2.start();
```

- Initializes the shared resource and starts both threads.
-

What Happens at Runtime

1. MS Word locks "Printer" and sleeps.
2. MS Excel locks "Scanner" and sleeps.
3. After sleeping:
 - MS Word tries to lock "Scanner" (already locked by MS Excel)
 - MS Excel tries to lock "Printer" (already locked by MS Word)

Result: Deadlock

- Both threads wait forever on each other's locks.
- No further progress is made.

How to Fix It

Solution: Lock Resources in Same Order

Change both threads to always lock res1 first, then res2.

For example:

```
// In both Process1 and Process2: synchronized(cr.res1) {  
    synchronized(cr.res2) {  
        // Safe block  
    }  
}
```

This removes the possibility of circular wait, thus preventing deadlock.

Best Practices

- Avoid acquiring multiple locks whenever possible.
- If using multiple locks, always lock them in the same order.
- Release locks as soon as possible.
- Use timeouts when acquiring locks.